21. Design Real-Time Location Tracking

Key Terms:

Term	Full Name	What it does
GPS	Global Positioning System	Satellite-based location (10m accuracy)
Geohash	Geographic Hash	Encodes lat/lng into hierarchical grid cells
GEORADIUS	Redis Geo Command	Finds all points within radius of location
Geofence	Geographic boundary	Virtual perimeter triggering enter/exit events
R-tree	Rectangle tree	Spatial index for efficient polygon queries
SSE	Server-Sent Events	One-way server-to-client streaming over HTTP
TimescaleDB	Time-series database	PostgreSQL extension optimized for time-series

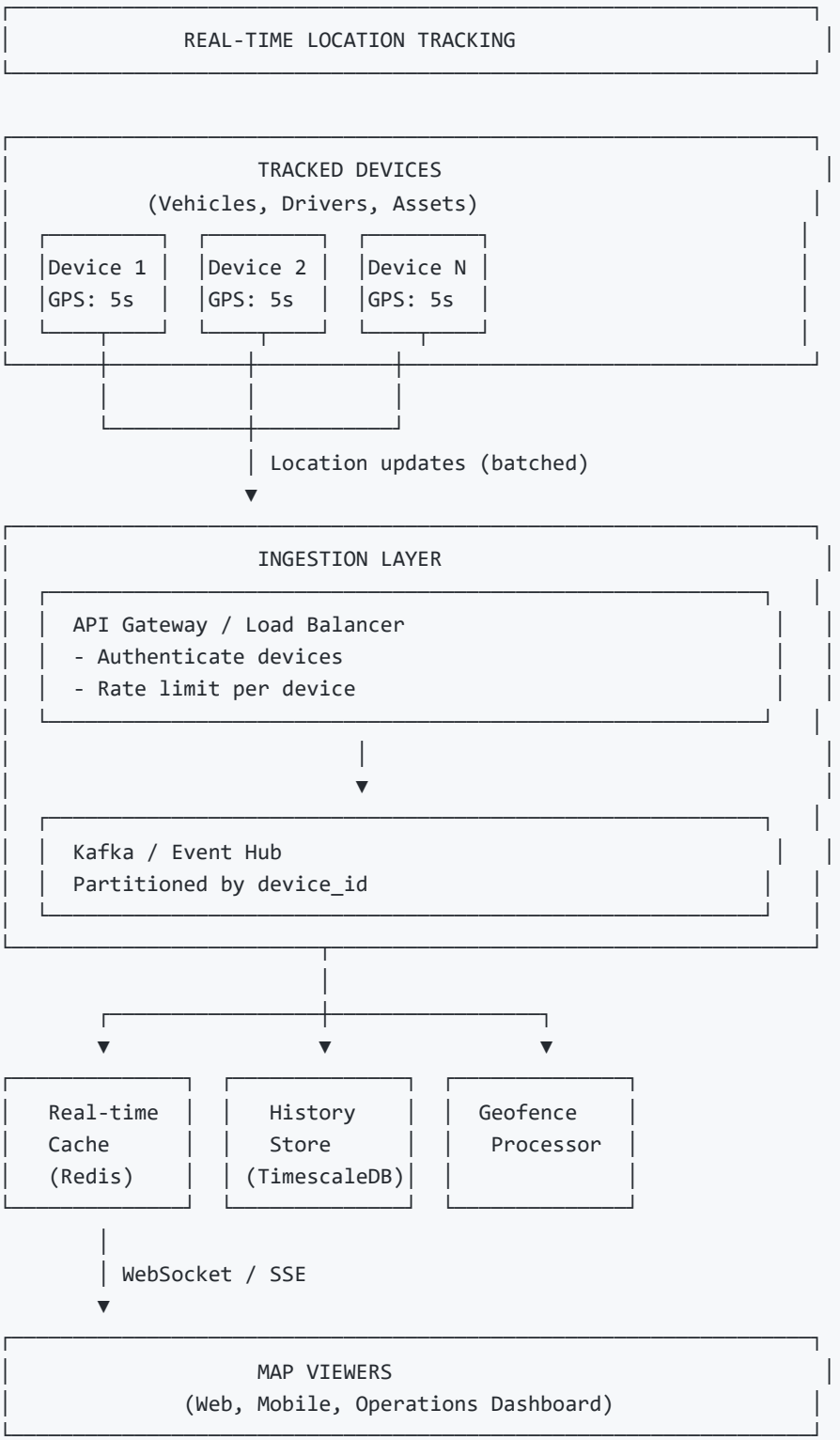
Problem Statement

Design a system that tracks and displays real-time locations of moving objects (vehicles, delivery drivers, assets) on a map.

Clarifying Questions to Ask

1. **Scale**: How many tracked objects? (1M vehicles?)
2. **Update frequency**: How often to update? (Every 5 seconds?)
3. **Precision**: GPS accuracy needed? (10 meters?)
4. **Features**: Historical tracking, geofencing, ETA?
5. **Viewers**: How many simultaneous map viewers?
6. **Battery**: Mobile devices need power optimization?
7. **Offline**: Handle intermittent connectivity?

High-Level Design



API Design

Location APIs

Method	Endpoint	Description

`PUT`	`/api/v1/locations/{entityId}`	Update entity location
`GET`	`/api/v1/locations/{entityId}`	Get current location
`GET`	`/api/v1/locations/{entityId}/history?from={t1}&to={t2}`	Get location history
`GET`	`/api/v1/locations/nearby?lat={lat}&lng={lng}&radius={m}`	Find nearby entities

Geofence APIs

Method	Endpoint	Description
`POST`	`/api/v1/geofences`	Create a geofence
`GET`	`/api/v1/geofences/{fenceId}`	Get geofence details
`DELETE`	`/api/v1/geofences/{fenceId}`	Delete a geofence

Location Update (High Frequency - via WebSocket/UDP)

```
{
  "entity_id": "driver_123",
  "lat": 47.6062,
  "lng": -122.3321,
  "accuracy": 10,
  "speed": 45.2,
  "heading": 270,
  "timestamp": 1672531200000
}
```

WebSocket Events

Event	Direction	Payload
`location.updated`	Server → Client	`{ entityId, lat, lng, speed }`
`geofence.enter`	Server → Client	`{ entityId, fenceId, timestamp }`
`geofence.exit`	Server → Client	`{ entityId, fenceId, timestamp }`

Key Components

Component	Purpose	Technology
Device SDK	Collect & send GPS	Mobile SDK
Ingestion API	Receive location updates	REST / gRPC
Message Queue	Buffer and distribute	Kafka
Real-time Cache	Current locations	Redis (Geospatial)
History Store	Historical tracking	TimescaleDB
Geofence Engine	Enter/exit alerts	Stream processor
Map Service	Render locations	WebSocket + Mapbox

Location Update Protocol

```
DEVICE TO SERVER:
┌──────────────────────────────────────────────────────────────┐
│  Batch updates to reduce battery and bandwidth:              │
│                                                               │
│  POST /locations                                             │
│  {                                                           │
│    "device_id": "vehicle_123",                               │
│    "locations": [                                            │
│      {"lat": 47.6062, "lng": -122.3321, "ts": 1705312340},   │
│      {"lat": 47.6065, "lng": -122.3318, "ts": 1705312345},   │
│      {"lat": 47.6068, "lng": -122.3315, "ts": 1705312350}    │
│    ],                                                        │
│    "battery": 85,                                            │
│    "accuracy": 10                                            │
│  }                                                            │
│                                                               │
│  Adaptive frequency:                                         │
│    - Moving fast: every 3 seconds                            │
│    - Moving slow: every 10 seconds                           │
│    - Stationary: every 60 seconds                            │
└──────────────────────────────────────────────────────────────┘
```

Geospatial Indexing

```
REDIS GEOSPATIAL:
┌──────────────────────────────────────────────────────────────┐
│  GEOADD vehicles <lng> <lat> vehicle_123                     │
│  GEOADD vehicles <lng> <lat> vehicle_456                     │
│                                                               │
│  Find vehicles within 5km of point:                          │
│  GEORADIUS vehicles -122.33 47.60 5 km WITHCOORD WITHDIST     │
```

```
│                                                                 │
│  Result:                                                        │
│  1) vehicle_123, 0.5km, (47.6062, -122.3321)                    │
│  2) vehicle_456, 2.1km, (47.5890, -122.3100)                    │
└─────────────────────────────────────────────────────────────────┘

GEOHASH FOR PARTITIONING:
┌─────────────────────────────────────────────────────────────────┐
│  Geohash divides world into grid cells                          │
│                                                                 │
│  47.6062, -122.3321 → geohash "c23nb"                           │
│                                                                 │
│  Subscribe to updates in region:                                │
│  SUBSCRIBE location:c23nb                                       │
│                                                                 │
│  Publish when vehicle moves:                                    │
│  PUBLISH location:c23nb {vehicle_123, lat, lng}                 │
└─────────────────────────────────────────────────────────────────┘
```

Geofencing

```
GEOFENCE DEFINITION:
┌─────────────────────────────────────────────────────────────────┐
│  {                                                              │
│    "fence_id": "warehouse_zone",                                │
│    "type": "polygon",                                           │
│    "coordinates": [[lat1,lng1], [lat2,lng2], ...],              │
│    "triggers": ["enter", "exit"],                               │
│    "actions": ["notify", "log"]                                 │
│  }                                                              │
└─────────────────────────────────────────────────────────────────┘

PROCESSING:
┌─────────────────────────────────────────────────────────────────┐
│  For each location update:                                      │
│    1. Find geofences near current location                      │
│    2. Check point-in-polygon                                    │
│    3. Compare with previous state                               │
│    4. If state changed → trigger event                          │
│                                                                 │
│  Optimization:                                                  │
│    - Spatial index for geofences (R-tree)                       │
│    - Cache previous state per device                            │
│    - Skip check if location change < threshold                  │
└─────────────────────────────────────────────────────────────────┘
```

Scalability Considerations

- **Ingestion**: Kafka partitioned by device_id for parallel processing
- **Real-time cache**: Redis Cluster with geospatial indexing

- **Map viewers**: WebSocket with geohash-based subscriptions
- **History**: TimescaleDB with time-based partitioning
- **Battery**: Adaptive update frequency based on movement

Trade-offs

Decision	Option A	Option B	Recommendation
Protocol	REST	gRPC	**REST** - simpler, HTTP/2 for batching
Updates	Real-time	Batched	**Batched** - battery efficient
Storage	SQL	Time-series DB	**TimescaleDB** - optimized for time-series
Push	Polling	WebSocket	**WebSocket** - real-time updates

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Kafka	High-throughput ingestion of location updates	Tech Guide
Redis	Geospatial indexing with GEOADD/GEORADIUS commands	Tech Guide
Geohash	Spatial partitioning for region-based subscriptions	Tech Guide

How to Explain It Well

> **Opening**: "Location tracking needs efficient ingestion and real-time delivery. I'll design for 1M devices updating every 5 seconds."

> **Key talking points**:
>
> - "Batch location updates from devices - reduces battery by 70%"
> - "Redis GEOADD/GEORADIUS for O(log N) spatial queries"
> - "Geohash-based subscriptions: viewers only get updates in their viewport"
> - "Adaptive frequency: fast updates when moving, slow when stationary"
> - "Geofence events processed in stream with R-tree spatial index"
